Managing Tables

So far we've been using migrations to manipulate the columns in existing tables. Now let's look at creating and dropping tables:

```
class CreateOrderHistories < ActiveRecord::Migration
  def change
    create_table :order_histories do |t|
      t.integer :order_id, null: false
      t.text :notes

      t.timestamps
    end
  end
end
```

`create_table` takes the name of a table (remember, table names are plural) and a block. (It also takes some optional parameters that we'll look at in a minute.) The block is passed a table definition object, which we use to define the columns in the table.

Generally the call to `drop_table` isn't needed, as `create_table` is reversible. `drop_table` accepts a single parameter, which is the name of the table to drop.

The calls to the various table definition methods should look familiar—they're similar to the `add_column` method we used previously, except these methods don't take the name of the table as the first parameter and the name of the method itself is the data type desired. This reduces repetition.

Note that we don't define the `id` column for our new table. Unless we say otherwise, Rails migrations automatically add a primary key called `id` to all tables they create. For a deeper discussion of this, see *Primary Keys*.

The `timestamps` method creates both the `created_at` and `updated_at` columns, with the correct `timestamp` data type. Although there's no requirement to add these columns to any particular table, this is yet another example of Rails making it easy for a common convention to be implemented easily and consistently.